

MLE5004: Computer systems architecture

VII. Instructions

Contents

7.1	Data management	1
7.1.1	MOV instructions	1
7.1.2	Load Effective Address	2
7.1.3	Exchange	3
7.1.4	Translate - XLAT	3
7.1.5	Flag instructions	4
7.2	Stack operations	4
7.2.1	Stack structure	5
7.2.2	Additional Push Instructions	6
7.2.3	Additional Pop Instructions	8
7.3	Type conversion	8
7.4	Branching, jumps, loops	8
7.5	Repetitive instructions	10
7.6	CALL and RET instructions	10

7.1 Data management

7.1.1 MOV instructions

What follows are some of the basic move instructions involving data transfer. These include register-to-register, immediate-data-to-register, immediate-data-to-memory, memory-to-register, and register-to-memory. Also included are moves with sign extension, moves with zero extension, and conditional moves that move data to a destination depending on the state of a flag. Data transfer instructions have the following general syntax:

MOV destination, source

The source is a general-purpose register, a segment register, immediate data, or a memory operand. The destination is a general-purpose register, a segment register, or a memory location.

General MOV instruction Examples of different forms of the MOV instruction are shown in Table 1. The general move instruction, MOV, cannot be used for memory-to-memory data transfer. The move string (MOVS) instruction is used for that operation. The MOV instruction also cannot move data from one segment register to another segment register.

Table 1: Different forms of the MOV instruction.

Register-to-register	MOV	EAX, EBX	
Immediate-to-register	MOV	AX, 1234H	
Direct	MOV	MEM_ADDR, AX	MEM_ADDR gives the address directly
Register indirect	MOV	EAX, [EBP]	[EBP] gives the address indirectly
Indexed	MOV	AX, [DI/SI + displacement]	Displacement may be 0
Based	MOV	EAX, [EBP/EBX + displacement]	Displacement may be 0
Base plus index	MOV	[BP/BX + SI/DI], AX	
Base plus index plus displacement	MOV	[BX/BP + DI/SI + displacement], EAX	

If the operand type is not obvious, then the type can be specified by either `BYTE PTR`, `WORD PTR`, `DWORD PTR`, for example, as shown below, which moves `0C16` to a memory location specified by the sum of the contents of registers `EBX` and `ESI`.

```
MOV BYTE PTR [EBX + ESI], 0CH
```

Move with Sign/Zero Extension The move with sign extension (*MOVSX*) instruction moves the source operand into a destination general-purpose register, then extends the sign bit into the high-order bits of the destination. The sign bit is extended into a 16-bit or a 32-bit destination. The *MOVSX* instruction is used with signed operands.

The move with zero extension (*MOVZX*) instruction is similar to the *MOVSX* instruction, except that zeroes are extended in the destination instead of the sign bit. The syntax for the *MOVSX* and the *MOVZX* instructions is shown below.

```
MOVSX/MOVZX register, register/memory/immediate
```

7.1.2 Load Effective Address

The load effective address (*LEA*) instruction loads the offset (effective address) of a memory address (source operand) within a memory segment and stores it in a general-purpose register. No flags are affected by this instruction. The address calculation of the *LEA* instruction is similar to the calculation performed by the *MOV* instruction, but the address of the source operand is stored in the destination, not the contents of the source operand. The *LEA* instruction can also be used for unsigned integer arithmetic. The syntax for the *LEA* instruction is shown below.

```
LEA destination, source
```

For example, the instruction `lea eax, [v]` loads into `EAX` the offset of the variable `v`, the instruction equivalent to `mov eax, v`. But *LEA* has the advantage that the source operand may be an addressing expression (unlike the *MOV* instruction which allows as a source operand only a variable with direct addressing in such a case). For example, the instruction:

```
lea eax, [ebx+v-6]
```

s not equivalent to a single MOV instruction. The instruction

```
mov eax, ebx+v-6
```

is syntactically incorrect, because the expression `ebx+v-6` cannot be determined at assembly time.

By using the values of offsets that result from address computations directly (in contrast to using the memory pointed by them), LEA provides more versatility and increased efficiency: versatility by combining a multiplication with additions of registers and/or constant values and increased efficiency because the whole computation is performed in a single instruction, without occupying the ALU circuits, which remain available for other operations (while the address computation is performed by specialized circuits in BIU).

Listing 1 shows as an example how to multiply a number with 10.

Listing 1: Multiply a number with 10.

```

1 mov eax, [number]           ; eax <- the value of the variable number
2 lea eax, [eax * 2]          ; eax <- number * 2
3 lea eax, [eax * 4 + eax]    ; eax <- (eax * 4) + eax = eax * 5 = (number * 2) * 5

```

7.1.3 Exchange

The exchange (XCHG) instruction exchanges the contents of the source and destination operands. The operands can be two general-purpose registers (GPRs) or a GPR and a memory location. This is a simpler and more expeditious method of exchanging two operands than the method using the MOV instruction because no temporary storage location is required to hold one of the operands.

When an XCHG instruction is executed, the LOCK prefix asserts the processor's LOCK signal. In a multiprocessor environment, the LOCK prefix ensures that the processor maintains uninterrupted use of any shared memory. This prevents other threads from accessing the memory location while the instruction is executing. The LOCK prefix is applicable only to certain instructions and is asserted for the duration of the instruction that follows the LOCK prefix; that is, the LOCK prefix prepends the instruction to which it applies.

The syntax for the XCHG instruction is shown below, where the source and destination operands can be a register or a memory location, but cannot both be memory locations. The flags are not affected by the XCHG instruction.

XCHG destination, source

XCHG register/memory, register/memory

7.1.4 Translate - XLAT

The translate (XLAT) instruction uses the contents of register AL as an index into a translation table in memory that contains the translated byte of data. The data at the memory location addressed by the index is then stored in AL. The index value in register AL is treated as an unsigned integer, which is added to the contents of the base register (E)BX to obtain the base address of the translation table in the data.

The XLAT instruction permits the base address of the translation table to be specified with a symbol. The base address must be explicitly stored in registers DS:(E)BX before execution of the XLAT instruction.

Listing 2 translates a decimal value “number” between 0 and 15 into the ASCII code of the corresponding hexadecimal digit.

Listing 2: Example XLAT, translate from a value into ASCII.

```

1  segment data use32
2  ...
3  TabHexa db '0123456789ABCDEF'
4  numar resb 1
5
6  segment code use32
7  ...
8  mov ebx, TabHexa
9  mov al, numar
10 xlat      ; AL <- < DS:[EBX+AL] >

```

7.1.5 Flag instructions

The following four instructions are flags transfer instructions:

- *LAHF* (Load register AH from Flags) copies SF, ZF, AF, PF and CF from FLAGS register in the bits 7, 6, 4, 2 and 0, respectively, of register AH. The contents of bits 5, 3 and 1 are undefined. Other flags are not affected (meaning that LAHF does not generate itself other effects on some other flags – it just transfers the flags values).
- *SAHF* (Store register AH into Flags) transfers the bits 7, 6, 4, 2 and 0 of register AH in SF, ZF, AF, PF and CF respectively, replacing the previous values of these flags.
- *PUSHF* transfers all the flags on top of the stack (the contents of the EFLAGS register is transferred onto the stack). The values of the flags are not affected by this instruction.
- *POPF* instruction extracts the word from top of the stack and transfer its contents into the EFLAGS register.

The assembly language provides the programmer with some instructions to set the value of the flags (the condition indicators) so that the programmer can influence the operation mode of the instructions which exploits these flags as desired.

- CLC, CF=0
- CMC, CF = CF
- STC, CF=1
- CLD, DF=0
- STD, DF=1

7.2 Stack operations

The stack is a one-dimensional data structure located in contiguous locations of memory that is used for the temporary storage of data. It is one of the segments in a segmented memory model and is called the stack segment, in which the base address of the stack is contained in the stack segment (SS) register. A stack can have a maximum size of four gigabytes. A general-purpose register called the stack pointer (E)SP contains the address of the current top of stack. A stack is comparable to a stack of trays in a cafeteria in which each tray is stacked on top of the last tray; the tray on the top of the stack is the tray that is normally removed first.

Generally, only the double word at the top of stack — pointed to by the stack pointer — is accessed; however, another GPR can also be used to access elements within the stack. This is the base pointer (E)BP register, which contains an offset into the stack segment to reference data or

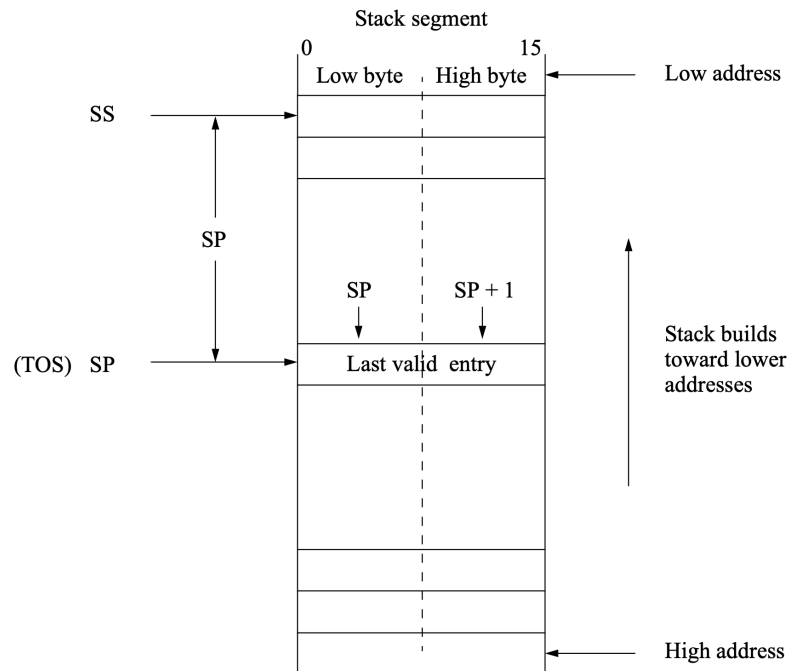


Figure 1: Caption

parameters that were passed to a called program via the stack. The default segment for the (E)BP register is the stack segment. Since the element at the top of the stack is removed first, stacks are referred to as a last-in-first-out (*LIFO*) queue.

A data element is placed on top of the stack by a **PUSH** instruction; a data element is removed from the top of the stack by a **POP** instruction. **A stack builds toward lower addresses.** When a data item is pushed onto the stack, the (E)SP register is first decremented, then the data item is stored at the new top of stack. When a data item is popped off the stack, it is first stored in the destination address, then the (E)SP register is incremented to point to the new top of stack. The operand size determines the amount that the stack pointer is incremented or decremented. For example, if the operand size is 16 bits, then the SP register is incremented/decremented by 2; if the operand size is 32 bits, then the ESP register is incremented/decremented by 4.

7.2.1 Stack structure

Figure 1 shows a drawing of a stack structure where the operand size is 16 bits; therefore, the SP register is used as the stack pointer. The structure is similar for an operand size of 32 bits where the items stored on the stack are doublewords; in this case, the stack pointer is the ESP register. In Figure 1, a PUSH operation first decrements SP by 2, then stores the operand at the new stack top, as shown below, where the colon (:) indicates concatenation.

Let us illustrate the way in which these instructions work, by using an equivalent sequence of MOV and ADD or SUB instructions.

```

push eax ⇔  sub esp, 4      ; prepare (allocate) space in order to store the value
             mov [esp], eax ; store the value in the allocated space

pop  eax ⇔  mov eax, [esp]  ; load in eax the value from the top of the stack
             add esp, 4     ; clear the location

```

In the perspective of evaluating the effect of instructions such as PUSH ESP or POP dword

[ESP], the order in which the component (sub)operations of the PUSH and POP instructions are performed should be specified even more clearly:

1. The source operand of the instruction is evaluated (ESP for PUSH and dword [ESP] respectively for POP)
2. ESP is updated accordingly ($ESP := ESP - 4$ for PUSH and $ESP := ESP + 4$ for POP respectively)
3. The assignment involved in the effect of the instruction is performed on the destination operand (the new top of the stack for PUSH and dword [ESP] respectively (this being now after the subtraction of ESP from b) the element immediately below the initial top of the stack - for POP)

Assuming that the initial situation is $ESP = 0019FF74$, after `PUSH ESP` we will have $ESP = 0019FF70$ and the contents of the top of the stack will now be `0019FF74`. Assuming that the initial situation is $ESP = 0019FF74$ and that in this location is the value `7741FA29` (the top of the stack), after `POP dword [ESP]` we will have $ESP = 0019FF78$ and the content of this location (the content of the location at the top of the stack) will be `7741FA29` (so we could say that "the top of the stack moves one position lower").

Figure 2 shows an example of pushing and popping 16-registers using the stack configuration of Figure 1. Prior to the operations, the general-purpose register, AX, is assigned a value of `1234h`; the stack segment register, SS, is assigned a value of `1050h`; and the stack pointer register, SP, is assigned a value of `0008h`. Figure 2a shows the initialization of the stack and registers. Figure 2b shows the `PUSH AX` operation; Figure 2c shows the `POP AX` operation; and Figure 2d shows the `POP BX` operation. The stack pointer is incremented or decremented accordingly for each operation. Flags are not affected by the PUSH and POP operations.

7.2.2 Additional Push Instructions

Push all 16-bit general-purpose registers (PUSHA) The PUSHA instruction is used to push all general-purpose registers (GPRs) onto the stack. If the operand-size attribute is 16, then the registers are pushed onto the stack in the following order: AX, CX, DX, BX, SP, BP, SI, and DI. Note that the value in SP that is pushed onto the stack is the value in SP before the PUSHA instruction is executed.

When a procedure call is executed, the processor does not automatically save the GPRs onto the stack prior to executing the call. The PUSHA instruction is one way that the calling/called procedure can save all the GPRs by executing a single instruction. The called procedure can also save only the specific registers that will be used before executing the procedure; the registers are then restored to their original values prior to returning to the calling program. No flags are affected by the PUSHA instruction.

Push all 32-bit general-purpose registers (PUSHAD) The instruction that pushes all 32-bit GPRs onto the stack is the PUSHAD instruction. If the operand-size attribute is 32 bits, then the registers are pushed onto the stack in the following order: EAX, ECX, EDX, EBX, ESP, EBP, ESI, and EDI. Note that the value in ESP that is pushed onto the stack is the value in ESP before the PUSHAD instruction is executed. The PUSHA and PUSHAD instructions have the same operation code and their operation is identical except for the registers that are pushed onto the stack, which are a function of the operand-size attribute.

When a procedure is called, saving the GPRs is identical as described under the PUSHA heading. It should be noted that some X86 assemblers may cause the operand size to be 16 bits for a PUSHA instruction or to be 32 bits when a PUSHAD instruction is encountered. Other X86 assemblers use the current value of the operand-size attribute. No flags are affected by the PUSHAD instruction.

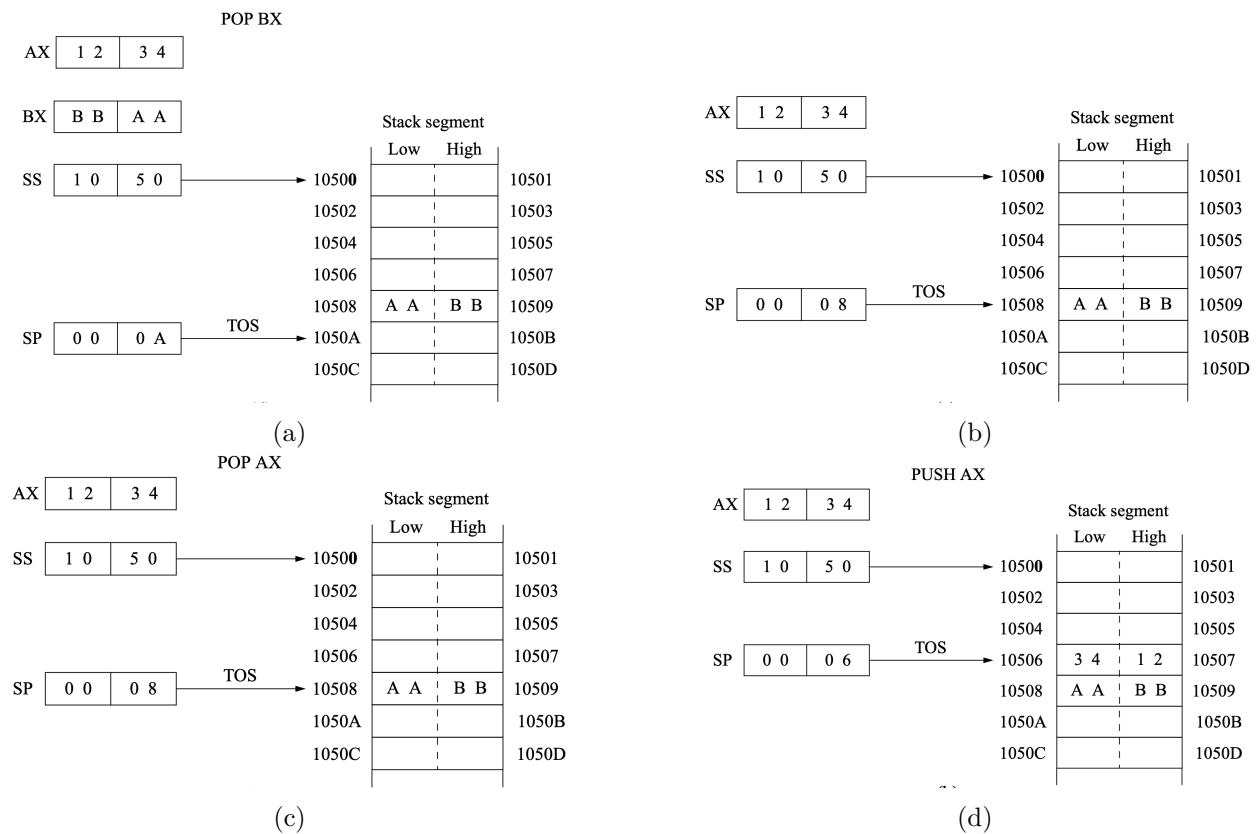


Figure 2: Diagrams showing stack operations for PUSH and POP instructions involving the general-purpose registers AX and BX on 16 bits: (a) initialization, (b) PUSH AX, (c) POP AX, and (d) POP BX.

7.2.3 Additional Pop Instructions

Pop all 16-bit general-purpose registers (POPA) The POPA instruction is used to pop all general-purpose registers off the stack. If the operand-size attribute is 16, then the word registers are popped off the stack in the following order and stored in their respective registers: DI, SI, BP, the pushed SP is ignored, BX, DX, CX, and AX. Note that the value in SP is discarded, because if SP were popped off the stack, then the value in the current SP register would be changed.

The POPA instruction is one way that the called/calling procedure can restore all the GPRs by executing a single instruction. The called procedure can also restore only the specific registers that were used in the procedure; the registers are then restored to their original values prior to returning to the calling program. The POPA instruction effectively reverses the operation of the PUSHA instruction. No flags are affected by the POPA instruction.

Pop all 32-bit general-purpose registers (POPAD) The POPAD instruction pops all 32-bit general-purpose registers off the stack. If the operand-size attribute is 32 bits, then the registers are popped off the stack in the following order: EDI, ESI, EBP, the pushed ESP is ignored, EBX, EDX, ECX, and EAX. Note that the value in ESP is discarded for the same reason that the value of SP was discarded for the POPA instruction. The POPA and POPAD instructions have the same operation code and their operation is identical except for the registers that are popped off the stack, which are a function of the operand-size attribute.

7.3 Type conversion

- **CBW** - converts the byte from AL to the word in AX (sign extension).
- **CWD** - converts the word from AX to the doubleword in DX:AX (sign extension).
- **CWDE** - converts the word from AX to the doubleword in EAX (sign extension).
- **CDQ** - converts the doubleword from EAX to the quadword in EDX:EAX (sign extension).
- **MOVZX d, s** - loads in d (REGISTER !), which must be of size larger than s (reg/mem), the UNSIGNED contents of s (zero extension).
- **MOVSX d, s** - load in d (REGISTER !), which must be of size larger than s (reg/mem), the SIGNED contents of s (sign extension).

Listing 3 shows some examples.

7.4 Branching, jumps, loops

Unconditional jump Three instructions fall into this category: JMP (equiv. to GOTO from other languages), CALL (a procedure call means a control transfer from the call's point to the first instruction from the called routine) and RET (control transfer back to the first executable instruction after the CALL).

- **JMP operand** - Unconditional jump to the address specified by operand.
- **CALL operand** - Transfers control to the procedure identified by operand.
- **RET [n]** - Transfers control to the first instruction after CALL.

Listing 3: Type conversions.

```

1  mov al, -1    ; AL=0FFh
2  cbw          ; extends the byte value -1 from AL to the
3              ; word value -1 in AX (0FFFh).
4
5  mov ax, -10000 ; AX = 0D8F0h
6  cwd          ; obtains the value -10000 in DX:AX
7              ; (DX = 0FFFFh ; AX = 0D8F0h)
8  cwde         ; obtains the value -10000 in EAX
9              ; EAX = 0FFFFD8F0h
10
11 MOV ah, 0c8h
12 MOVsx ebx, ah ; EBX = FFFFFFFC8h
13 MOVZX edx, ah ; EDX = 000000C8h
14 MOVsx ax, [v] ; MOVsx ax, byte ptr DS:[offset v]
15 MOVZX eax, [v] ; syntax error - op.size not specified
16 Movsx eax, v   : syntax error, v is not reg/mem as the
17              ; syntax requires, but a constant

```

Conditional jump instructions

- **CMP d,s** - compares the operands values (does not modify them - fictitious subtraction $d - s$), flags affected: OF, SF, ZF, AF, PF and CF.
- **TEST d,s** - non-destructive, **d AND s**, flags affected: OF = 0, CF = 0, SF, ZF, PF - modified, AF - undefined.

Conditional jump instructions are usually used combined with comparison instructions. Thus, the semantics of jump instructions follows the semantics of a comparison instruction. Besides the equality test performed by a **CMP** instruction we need frequently to determine the exact order relationship between 2 values. For example we have to answer to: nr. 11111111b (= FFh = 255 = -1) is bigger than 00000000b (= 0h = 0)? The answer is it depends! This answer can be either YES or NO! If we perform an unsigned comparison, then the first one is 255 and is obvious bigger than 0. If the 2 values are compared in the signed interpretation, then the first is -1 and is less than 0.

The **CMP** instruction does not make any difference between the two above cases, because addition and subtraction are performed always in the same way (adding or subtracting binary configurations) no matter their interpretations (signed or unsigned). So it's not the matter to interpret the operands of **CMP** as being signed or unsigned, but to further interpret the result of the subtraction! Conditional jump instructions are responsible to do that.

For all the conditional jump instructions the general syntax is

<conditional_jump_instruction> label

The effect of the conditional jump instructions is expressed as "jump if operand1 «relationship» operand2" (where on the two operands a previously **CMP** or **SUB** instruction is supposed to have been applied) or relative to the actual value set for a certain flag. As easy can be noticed based on the conditions that must be verified, instructions on the same line in the table have similar effect.

When two signed numbers are compared, "less than" and "greater than" terms are used and when two unsigned numbers are compared "below" and "above" terms are respectively used.

7.5 Repetitive instructions

These are: LOOP, LOOPE, LOOPNE and JECXZ. Their syntax is

`<instruction> label`

LOOP performs the repetitive run of the instructions block starting at label, as long as the value of CX register is different from 0. It first performs decrementation of ECX, then the test and eventually the jump. The jump is "short" (max. 127 bytes – so pay attention to the "distance" between LOOP and the label!).

When the end of loop conditions are more complex LOOPE and LOOPNE may be used. LOOPE (LOOP while Equal) differ from LOOP by ending condition, loop is ended either if ECX=0, either if ZF=0. In the case of LOOPNE (LOOP while Not Equal) the loop will end either if ECX=0, either if ZF=1. Even if the loop exit shall be based on value of ZF, CX decrementation is done anyway. LOOPE is also known as LOOPZ and LOOPNE is also known as LOOPNZ. These are usually used preceded by a CMP or SUB instruction.

JECXZ (Jump if ECX is Zero) performs the jump to the operand label only if ECX=0, being useful when we want to test the value in ECX before entering in a loop. In the following example, JECXZ instruction is used to avoid entering the loop if ECX=0.

```

1  jecxz MaiDeparte      ;if ECX=0 a jump over the loop is made
2  Bucla:
3      Mov BYTE [esi],0  ;initializing the current byte
4      inc si            ;passing to the next byte
5      loop Bucla        ;resume the loop or ending it
6  MaiDeparte:
7  ...

```

If a loop is entered with ECX=0, ECX is first decremented, obtaining the value 0FFFFFFFFh (= -1, so a value different from 0), the loop being resumed until 0 in ECX will be reached, namely 2³² more times!

None of the presented repetitive instructions affects the flags.

7.6 CALL and RET instructions

A procedure call is done by using the CALL instruction, it can be a direct or an indirect call. The direct call has the syntax:

`CALL operand`

Similar to JMP, CALL transfers the control to the address specified by the operand. In addition to JMP, before performing the jump, CALL saves to the stack the address of the instruction following CALL (the returning address). In other words, we have the equivalence:

1	<code>CALL operand</code>		<code>push dword A</code>
2	<code>A: . . .</code>		<code>jmp operand</code>

The end of the called sequence is marked by a RET instruction. This pops from the stack the returning address stored there by CALL, transferring the control to the instruction from this address. The RET syntax is:

`RET [n]`

where n is an optional parameter. It indicates freeing from the stack n bytes below the returning address.

RET instruction can be illustrated by this equivalence:

```

1          B    resd 1
2 RET n      .      .      .
3 (near return) <=> pop dword [B]
4             add esp,n
5             jmp  [B]
```

Usually, as it is natural, CALL and RET are used in the following context:

```

1 procedure_label:
2 .      .      .
3     ret  n
4 .      .      .
5 CALL procedure_label
```

CALL may also take the transfer address from a register or from a memory variable. Such a call is identified as an indirect call. Example:

```

1 procedure_label:
2 call ebx      ;address taken from a register
3 call [vptr]   ;address taken from a memory variable
```

The destination operand of a CALL instruction may be: a procedure name, the name of a register, containing an address, a memory address.

References

1. A Vancea. Courses, (tank you :))
2. Cavanagh, J. (2013). X86 Assembly Language and C Fundamentals (1st ed.). CRC Press. <https://doi.org/10.1201/b14582>
3. Randall Hyde. (2020). The Art of Assembly Language (2nd Edition). No Starch Press.
4. Daniel Kusswurm. (2023). Modern X86 Assembly Language Programming: 32-bit, 64-bit, SSE, and AVX. Apress.